

Asymmetric Residual Adapters for Domain-Incremental Audio Classification

Aakash Divakar*, Angad Ripudaman Singh Bajwa*, Atharva Anand Joshi*

Independent Researcher, Austin, USA

{aakashdivakar.4869, angadisthere, atharva.joshi253}@gmail.com

Abstract—Domain-incremental learning (DIL) requires a single model to classify a fixed set of sound classes as the recording conditions change across a sequence of domains, without access to earlier-domain data and without forgetting. We study this problem in the setting of DCASE 2026 Task 7. Starting from the released baseline, which adds per-domain Batch Normalization (BN) and a per-domain classifier to a shared, frozen CNN14 backbone, we identify limited per-domain capacity as the main bottleneck on the harder domains. We insert, on top of the frozen backbone, a stack of zero-initialised *residual bottleneck adapters* after every convolutional block, and we set the stack depth *per domain* so that a domain receives capacity in proportion to how far it drifts from the frozen backbone. Combined with a warm-started per-domain head and a supervised contrastive loss, this raises the domain-agnostic average accuracy on the development set from 50.0% to 68.5%, while reproducing the first domain bit-for-bit so nothing is forgotten. To test generality we build a second DIL benchmark from TAU Urban Acoustic Scenes 2019, with four cities forming the domain sequence, and the same recipe again improves on the ADIL baseline across the sequence. Because these cities are balanced in difficulty, an equal (*symmetric*) depth is best here, whereas Task 7’s uneven domains call for an asymmetric one. Both follow a single rule: give each domain adapter capacity in proportion to its difficulty. Finally, we compare three domain-routing strategies and find the parameter-free entropy router the most robust: learned routers overfit the development split, and, on genuinely unseen domains (ESC-50), entropy routing mis-routes with high confidence rather than abstaining, a failure mode we argue is more consequential for deployment than raw accuracy.

Index Terms—Domain-incremental learning, continual learning, acoustic classification, residual adapters, catastrophic forgetting

1. INTRODUCTION

Domain-incremental learning (DIL) requires a single model to classify the same set of sound classes as the recording conditions change over a sequence of domains $\mathcal{D}_1 \rightarrow \mathcal{D}_2 \rightarrow \mathcal{D}_3$, without access to the data of previous domains and without forgetting them [1]. Naively fine-tuning the whole network on a new domain overwrites the knowledge of earlier domains (catastrophic forgetting), while freezing the network entirely prevents it from adapting (poor plasticity). This stability-plasticity trade-off is shared with the broader continual-learning literature, but DIL differs from class-incremental learning in that the label space is fixed and only the acoustic conditions shift, and from unsupervised domain adaptation in that no source data is available once a new domain arrives.

The DCASE 2026 Task 7 baseline addresses this trade-off with the ADIL approach [1]: a CNN14 [2] backbone is trained on $\mathcal{D}_1$ and then *frozen*; for every subsequent domain only a small set of domain-specific parameters is learned, namely the affine Batch-Normalization (BN) parameters of that domain and a residual correction to the classifier. At inference the domain identity is not given, so the model forward-passes through every domain’s specific layers and selects the domain whose prediction has the lowest entropy (domain-agnostic, ADIL).

This design forgets nothing, since earlier-domain parameters are never touched, but its plasticity is limited: a handful of affine BN

parameters is too little capacity for a large shift, which is what we observe on the two harder development domains, where the reproduced baseline reaches only 59.2% (D2) and 40.8% (D3). Adapters, which add a few task-specific parameters to a frozen backbone and are widely used for efficient transfer in NLP and vision, are a natural way to add plasticity without disturbing learned weights; we adapt them to the forgetting-free setting of Task 7.

Our central claim is that per-domain *capacity*, rather than the per-domain classifier, is the bottleneck on the harder domains, and that this capacity should be allocated in proportion to how far each domain drifts from the frozen backbone. From this we build a forgetting-free extension of the baseline (Fig. 1) whose two core ingredients are (i) per-domain, zero-initialised residual bottleneck adapters inserted after every convolutional block, and (ii) a per-domain adapter *depth* set by difficulty, which the same rule makes asymmetric on Task 7 and symmetric on the balanced TAU benchmark. Two lighter additions, a warm-started per-domain head and a supervised contrastive loss, sharpen the per-domain decision boundaries. Every addition is per-domain and zero/identity-initialised, so $\mathcal{D}_1$ is reproduced exactly and the domains stay mutually independent. Alongside the method, we contribute a cautionary routing study: we compare three inference-time domain routers and show where confidence-based routing fails on genuinely unseen audio. This paper reports the full ablation and routing study behind our DCASE 2026 Task 7 submission.

2. RELATED WORK

Domain-incremental learning is one of three continual-learning settings of Van de Ven and Tolias [3], set apart by a label space that stays fixed while only the input distribution shifts. Classical remedies for the stability-plasticity trade-off regularize updates toward parameters important for earlier tasks (EWC [4]), distill an earlier checkpoint (LwF [5]), or replay stored examples (iCaRL [6]). None apply once a Task 7 domain boundary is crossed, where no earlier data or gradients remain, which is why ADIL [1] and our method freeze the shared backbone and isolate learning to small per-domain modules.

Parameter-efficient adaptation of a frozen backbone predates continual learning. Bottleneck adapters were popularized for NLP transfer by Houlsby et al. [7] and for multi-domain vision by Rebuffi et al. [8], whose residual adapters are the closest precedent to our per-domain bottleneck. The alternatives we benchmark in Table 2, FiLM [9] and squeeze-and-excitation [10], were built for visual reasoning and channel recalibration, not continual adaptation, and LoRA [11] targets larger models than CNN14, where the plain bottleneck sufficed. What sets our use apart is the zero initialization and per-domain stacking, which reproduce earlier domains exactly rather than approximately.

Detecting when a model runs outside its training distribution is usually studied apart from continual learning. Hendrycks and Gimpel [12] use maximum softmax probability as a weak out-of-distribution signal, closely related to the entropy routing we inherit from ADIL; Lee et al. [13] and Lakshminarayanan et al. [14], both central to

*The three authors contributed equally to this work.

our routing comparison, instead model the feature distribution or use ensemble disagreement. Our ESC-50 probe adds a case where prediction entropy fails silently rather than degrading gracefully, since a wrong but confident domain head stays peaked on genuinely unseen audio.

3. METHOD

We keep the baseline front-end and backbone unchanged. Audio is resampled to 32 kHz; a log-mel spectrogram with 64 mel bands is computed with a window of 1024 and a hop of 320 samples. The backbone is CNN14 [2]: six convolutional blocks producing $\{64, 128, 256, 512, 1024, 2048\}$ channels, each block being two 3×3 convolutions, each followed by BN and ReLU, and a 2×2 average pooling. Following the baseline, the convolutional weights are loaded from the released $\mathcal{D}_1$ checkpoint and *frozen for all domains*; only the BN layers are made domain-specific, i.e. each domain t owns its own BN parameters and statistics.

To give each domain more than the affine BN capacity, we insert a residual bottleneck adapter after each of the six convolutional blocks. For a feature map with C channels the adapter is

$$\text{Adapter}(\mathbf{x}) = \mathbf{x} + W_{\text{up}} \sigma(W_{\text{down}} \mathbf{x}), \quad (1)$$

where W_{down} is a 1×1 convolution reducing the channels to $C/4$, σ is a ReLU, and W_{up} is a 1×1 convolution restoring C channels. W_{up} is *zero-initialised*, so at the start of training every adapter is the identity map and the pretrained features are left untouched; the adapter then learns a domain-specific feature shift on top of the frozen backbone. Domain $\mathcal{D}_1$ uses identity adapters, so its behaviour is reproduced bit-for-bit.

Because each domain owns an independent stack of adapters, the *number* of adapters stacked after each block can be chosen per domain. We stack d_t residual adapters for domain t ,

$$\text{Stack}_t(\mathbf{x}) = (\text{Adapter}^{(d_t)} \circ \dots \circ \text{Adapter}^{(1)})(\mathbf{x}), \quad (2)$$

and every adapter is zero-initialised, so a stack of any depth still starts as the identity. The rule we follow is to give a domain capacity in proportion to how far it has drifted from the frozen backbone. On Task 7 the third domain is much harder than the second, so we set $d_{D2} = 1$ and $d_{D3} = 3$, giving D3 three times the per-domain convolutional capacity of D2; because the two stacks are independent, increasing D3’s depth cannot affect D2. When the domains are balanced in difficulty the same rule asks for equal depth, which is exactly what we find best on the TAU benchmark later, so the asymmetry is a consequence of per-domain difficulty rather than a fixed design choice.

The baseline reuses the frozen $\mathcal{D}_1$ classifier for all domains. We instead give every domain $t > 0$ its own linear head $\mathbf{W}_t \in \mathbb{R}^{C \times K}$ ($K = 10$ classes), warm-started from the $\mathcal{D}_1$ classifier at checkpoint-load time. $\mathcal{D}_1$ keeps the original head. The head is trained jointly with the domain’s BN and adapters.

Each domain is trained with cross-entropy plus a supervised contrastive auxiliary loss [15] applied to the L2-normalised global embedding $\mathbf{z}$:

$$\mathcal{L}_{\text{con}} = \sum_i \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_p / \tau)}{\sum_{a \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_a / \tau)}, \quad (3)$$

where $P(i)$ is the set of samples sharing i ’s class. The total loss is $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{con}}$ with $\lambda = 0.2$ and $\tau = 0.07$, which pulls same-class embeddings together and pushes different-class embeddings apart, sharpening the per-domain decision boundaries.

We also tried a training-only waveform augmentation pipeline: one clean view per 4 s segment plus eight augmented views (shift/stretch/speed/fade, class-aware pitch and tempo, spectral filtering, colored noise, a label-preserving distractor, and local masking/gain/clipping).

Since the domain identity of a test sample is unknown, we need a routing rule that selects which domain-specific layers to use. We compare three strategies on top of the same trained model.

The first strategy, entropy routing, requires no additional training. For each test sample we compute class probabilities under each domain head, $\mathbf{p}_t = \text{softmax}(\mathbf{l}_t)$, and select the domain with minimum predictive entropy [1]:

$$t^* = \arg \min_t H_t, \quad H_t = - \sum_c p_{tc} \log p_{tc}. \quad (4)$$

A well-matched domain head produces a peakier, lower-entropy distribution than a mismatched one, so t^* acts as an implicit domain detector, as predictive entropy does for confidence in deep ensembles [14].

The second strategy, a Mahalanobis distance router following Lee et al. [13], models each domain’s distribution in the D1-BN feature space $\mathbf{z} = g_0(\mathbf{x}) \in \mathbb{R}^{2048}$ (the GAP output of Conv Block 6 under frozen D1 statistics). From the D2 and D3 training embeddings we estimate diagonal-covariance Gaussians $(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ with a $\lambda = 10^{-4}$ diagonal regulariser via streaming statistics, and route by

$$t^* = \begin{cases} D1 & \text{if } \min(d_{D2}, d_{D3}) > \tau, \\ \arg \min_k d_k(\mathbf{z}) & \text{otherwise,} \end{cases} \quad (5)$$

with $d_k(\mathbf{z}) = \sqrt{(\mathbf{z} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{z} - \boldsymbol{\mu}_k)}$ and a rejection threshold $\tau = 60$ calibrated on dev-test, below which a sample is routed to D1 instead of D2/D3.

The third strategy is a learned GAP-MLP router. In the spirit of mixture-of-experts gating [16], [17], we concatenate the domain-specific GAP activations from all three heads ($3 \times 2048 = 6144$ -d) and train a Linear($6144 \rightarrow 256$)-ReLU-Linear ($256 \rightarrow 2$) MLP with cross-entropy on the labeled development split (Adam, 5 epochs, $\text{lr} = 10^{-3}$) to predict $\{D2, D3\}$; a sample is rejected to D1 if $\max(\text{softmax}) < \tau_{\text{MLP}} = 0.5$.

Unless noted otherwise, the submitted system uses entropy routing, which adds zero parameters and requires no domain labels at any stage.

4. EXPERIMENTS AND RESULTS

All systems train on the Task 7 development set one domain at a time from the released $\mathcal{D}_1$ checkpoint, with Adam ($\text{lr} 10^{-3}$), batch size 32, and CosineAnnealingLR decaying to 10^{-7} ; only the current domain’s parameters (BN, adapters, head) are updated. We report domain-agnostic accuracy on D2 and D3 and their average after the full $\mathcal{D}_1 \rightarrow \mathcal{D}_2 \rightarrow \mathcal{D}_3$ sequence on the dev-test split (D2: 639, D3: 806 samples). The submitted configuration trains for 180 epochs with $\lambda_{\text{con}} = 0.2$, $\tau = 0.07$, and depths (1, 3); the augmentation run below uses 240 epochs.

The rest of this section makes three points in turn. First, per-domain capacity, not the classifier, is the bottleneck, and a residual bottleneck adapter is the right way to add it (Tables 1, 2). Second, this capacity should track each domain’s difficulty, which we confirm on Task 7 and on a second benchmark built from TAU2019 (Tables 5, 6). Third, since the domain identity is unknown at test time, the capacity must be routed without labels, and here confidence-based routing fails on genuinely unseen audio (Tables 3, 7).

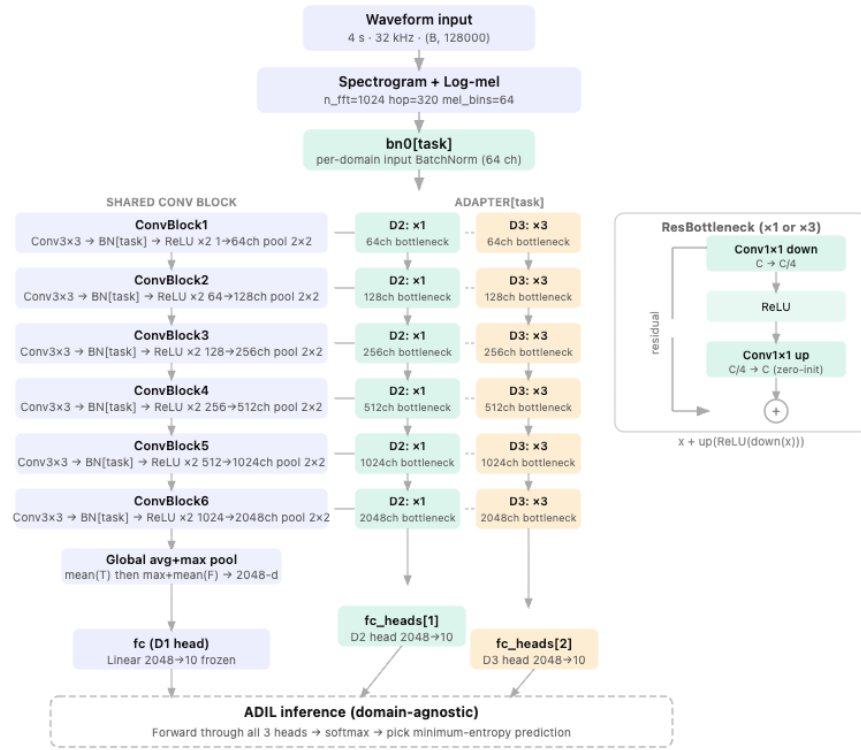


Fig. 1: Architecture of the submitted system. A shared, frozen CNN14 backbone (purple) is augmented with per-domain components: an input and per-block Batch Normalization, a stack of zero-initialised residual bottleneck adapters after every convolutional block (green for D2, amber for D3, with asymmetric depth 1 vs. 3), and a per-domain linear head. At inference the input is routed through all domains and the minimum-entropy prediction is kept (ADIL). The inset details one residual bottleneck adapter.

Table 1: Domain-agnostic per-domain accuracy (%) on the dev-test split, adding one component at a time. D2 and D3 are the accuracies on each domain’s own test data after the full sequence, and Avg is the average of the two. The baseline is our reproduction of the per-domain BN variant of ADIL [1] (BN only, without the classifier correction); the last row is the submitted system. The full ADIL system appears in Table 5.

System	D2	D3	Avg
Per-domain BN (baseline) [1]	59.2	40.8	50.0
+ residual adapters	73.9	53.5	63.7
+ per-domain head	77.3	54.0	65.6
+ supervised contrastive loss	77.9	55.1	66.5
+ asymmetric depth (1, 3)	77.2	57.7	67.4
+ longer training (180 ep.)	76.2	60.8	68.5

Table 1 adds one component at a time. Residual adapters give the largest single jump (+13.7 average), confirming that per-domain capacity, not the classifier, is the bottleneck. The per-domain head and contrastive loss mainly help D2, while asymmetric depth and longer training move the harder D3 (from 55.1% to 60.8%). The submitted system reaches 68.5% average, 18.5 points over the reproduced baseline, while preserving D1 exactly.

We compared three per-domain adapter families at the same six locations under the identical protocol (Table 2). FiLM, which only scales and shifts each channel, fell *below* the BN baseline, too weak to absorb the shift. Squeeze-and-excitation gating helped (57.7%) but did not improve with more blocks. The residual bottleneck, which mixes channels through its 1×1 convolutions, was strongest (63.7%) and is the one we keep, with the largest gain on the harder D3 (38.8 $\rightarrow$ 53.5%).

Table 2: Per-domain adapter family, same backbone and protocol (residual bottleneck selected).

Adapter family	D2	D3	Avg
FiLM (affine per channel)	56.7	38.8	47.7
Squeeze-and-excitation	70.3	45.0	57.7
Squeeze-and-excitation (deeper)	69.2	41.8	55.5
Residual bottleneck (<i>ours</i>)	73.9	53.5	63.7

Table 3: Routing strategy comparison on the dev-test split.

Router	D2	D3	Avg
Entropy (no threshold)	76.2	60.8	68.5
Entropy + ensemble ($\times 2$)	74.0	50.5	60.9
Mahal. (no threshold)	70.9	62.5	66.7
Mahal. ($\tau = 60$, diag)	68.5	57.7	63.8
GAP-MLP (3 ep., $\tau_{MLP} = 0.5$)	76.5	62.0	69.3

Table 3 compares the three routers on the best single model (asymmetric depth 1, 3, 180 epochs).

Entropy routing needs no training and benefits from the head calibration enforced by the contrastive loss. Mahalanobis routing adds a geometric model of the domains; without a rejection threshold it matches entropy routing (66.7%) at 73.2% D2 $\leftrightarrow$ D3 routing accuracy, but with $\tau = 60$ it rejects 22.8% of D2/D3 samples to D1 and the average *drops*, showing a global distance threshold is too coarse when the domains overlap. The learned GAP-MLP router reaches 69.3% at its best epoch but overfits quickly: on the *unlabeled* evaluation set it assigns only 0.6% of samples to D1, almost never rejecting, having memorised the dev-train distribution. We keep entropy routing as the default, since the learned routers add a calibration burden without a

Table 4: Partial backbone unfreezing for D3. No configuration beats the frozen adapter system; the shared weights couple D2 and D3.

Configuration	D2	D3	Avg
Frozen adapters (submitted)	76.2	60.8	68.5
Unfreeze 2 blk, no anchor	58.5	66.4	62.5
Unfreeze 2 blk, ℓ_2 anchor	75.3	57.3	66.3
Unfreeze 1 blk, ℓ_2 anchor	72.6	57.7	65.2
D3-only unfreeze 2 blk	61.5	62.3	61.9
D3-only unfreeze 3 blk	46.3	54.3	50.3

Table 5: Task 7 dev-test accuracy (%). D2 is the accuracy on D2, and Avg of D2,D3 is the average over D2 and D3. For single task, per-domain BN, and ADIL these are organizer-reported reference figures; ours matches the submitted run in Table 1. Single task trains a CNN14 from scratch per domain and cannot retain earlier domains.

System	D2	Avg of D2,D3
Single task	67.76	61.20
Per-domain BN (organizer)	58.60	52.50
ADIL [1]	76.06	64.50
Residual adapters (<i>ours</i>)	76.20	68.50

reliable held-out gain.

With augmentation (240 epochs) the system reaches 75.4/51.4/63.4% (D2/D3/avg), strong on D2 but losing on D3, where the broad augmentation pulls samples away from the characteristics the adapters must specialise for; we keep it as a robustness ablation only. Test-time augmentation and checkpoint ensembling were within noise and dropped.

Depth-2 stacks on *both* domains scored 65.1% versus 67.4% for the asymmetric (1, 3) at a similar budget, so capacity is better spent on the domain that needs it. The SupCon weight peaked at $\lambda = 0.2$ (66.7%); larger values hurt accuracy.

Unfreezing the top backbone blocks for D3 (Table 4) pushes D3 as high as 66.4%, our best D3, but because the backbone is shared D2 collapses to 58.5%. An ℓ_2 anchor to the previous checkpoint recovers D2 but cancels most of the D3 gain, and three blocks destabilise training. No unfrozen configuration beats the fully-frozen adapter system on average, since the shared weights couple the domains, so we keep the backbone frozen and rely on per-domain adapters, which add plasticity *without* cross-domain interference.

Table 5 places the adapters against three references on Task 7: a single-task model that trains a CNN14 from scratch on the current domain, and so cannot retain earlier ones, and the organizer-reported per-domain BN and full ADIL baselines. The adapters are highest both on D2 and on the sequence average, reaching 68.5% against 64.5% for full ADIL. Full ADIL here is the complete baseline, per-domain BN *plus* the classifier correction, and is well above the BN-only 50.0% starting point our adapters build on in Table 1.

All results so far are on Task 7. To test transfer we build a second DIL benchmark from TAU Urban Acoustic Scenes 2019 [18], with four cities forming the sequence $\mathcal{D}_1 \rightarrow \mathcal{D}_2 \rightarrow \mathcal{D}_3 \rightarrow \mathcal{D}_4$ over the same ten scene classes. We train the $\mathcal{D}_1$ backbone on a pool of six cities, freeze it, and apply the identical adapter recipe, comparing against the ADIL baseline retrained on the same splits.

The Task 7 pattern repeats in milder form. The adapters lead both references at the first, third, and fourth stages, ending at 72.46% against 68.7% (ADIL) and 67.33% (single task), trailing only on Lisbon where the extra capacity does not pay off. As before, they help most where a domain is hard; because the four cities are close in difficulty an equal symmetric depth is best, whereas Task 7’s steep gap favours an asymmetric one, so depth follows difficulty. Single task falls from 76.33% to 67.33%, unable to retain earlier cities. Per

Table 6: Second benchmark on TAU2019 cities, four cities forming the domain sequence (D1 a six-city pool, D2 Lisbon, D3 Lyon, D4 Prague). Each entry is the domain-agnostic accuracy averaged over the current city and all previously seen ones. The adapters use a symmetric depth here, which works best when the cities are close in difficulty.

System	D1	D2	D3	D4
Single task	76.33	69.21	66.71	67.33
ADIL [1]	67.3	69.6	71.4	68.7
Residual adapters (<i>ours</i>)	76.75	65.17	73.28	72.46

Table 7: Accuracy and routing on an unseen domain (ESC-50) vs. class-balanced in-domain references (7 classes, 40 clips/class).

Test set	Acc. (%)	Mean H	D1/D2/D3 (%)
ESC-50 (unseen)	85.0	0.027	14/53/33
D2 dev-test (ref.)	71.9	0.060	14/54/32
D3 dev-test (ref.)	55.7	0.027	19/34/47

class, the weak spot is airport, a confusable reverberant space, and the router occasionally misroutes park audio, the unreliability we examine next.

To probe generalisation beyond $\mathcal{D}_1\text{--}\mathcal{D}_3$, we run the submitted system, *with no further training*, on ESC-50 [19], recorded on entirely different sources and microphones. We use the 7 ESC-50 categories overlapping the Task 7 label space (alarm, baby_cry, dog_bark, engine, fire, footsteps, knocking; 40 clips/class), and compare against a matched 40-clip/class subsample of the same 7 classes from D2/D3 dev-test.

Two findings stand out. Accuracy on unseen ESC-50 clips is *higher* than on either in-domain reference, so the D2/D3 gap is not merely domain novelty. ESC-50’s single-source, high-SNR recordings are acoustically easier, and part of the D3 shortfall is intrinsic sample difficulty rather than missing adapter capacity. More importantly, entropy routing has no notion of “none of the above”, and ESC-50 is routed to D2/D3 with mean entropy (0.027) as low as the confident D3 reference. Only an explicit-rejection router, such as thresholded Mahalanobis distance, can express “neither D2 nor D3”, the more practical failure mode to address before deployment.

5. CONCLUSION

We presented a forgetting-free extension of the DCASE 2026 Task 7 baseline that raises per-domain capacity with zero-initialised residual adapters, allocating depth per domain so capacity follows difficulty, plus a warm-started per-domain head and a supervised contrastive loss. It lifts the dev-test average from 50.0% to 68.5% while reproducing $\mathcal{D}_1$ exactly, and transfers to a second benchmark from TAU2019 cities, where balanced difficulty makes a symmetric allocation best. Of the three routers, parameter-free entropy routing is the most robust: it needs no training and, unlike the learned GAP-MLP router, does not overfit the labeled split. Its one failure, shared by every confidence-based router, is that it cannot signal “none of the seen domains” and misroutes unseen audio confidently, as our ESC-50 probe shows. Adding an explicit rejection option, and closing the hard-domain gap without the cross-domain interference that unfreezing introduces, are the natural next steps.

REFERENCES

- [1] M. Mulimani and A. Mesaros, “Domain-incremental learning for audio classification,” in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, 2025, pp. 1–5.
- [2] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “PANNs: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 28, pp. 2880–2894, 2020.

- [3] G. M. van de Ven and A. S. Tolias, “Three scenarios for continual learning,” arXiv:1904.07734, 2019.
- [4] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Overcoming catastrophic forgetting in neural networks,” *Proc. National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [5] Z. Li and D. Hoiem, “Learning without forgetting,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, 2018.
- [6] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “iCaRL: Incremental classifier and representation learning,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017.
- [7] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019.
- [8] S.-A. Rebuffi, H. Bilen, and A. Vedaldi, “Learning multiple visual domains with residual adapters,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. Artif. Intell.*, 2018.
- [10] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.
- [11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [12] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [13] K. Lee, K. Lee, H. Lee, and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [14] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [15] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, “Supervised contrastive learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [16] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [17] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [18] T. Heittola, A. Mesaros, and T. Virtanen, “TAU Urban Acoustic Scenes 2019, Development dataset,” Zenodo, 2019, <https://doi.org/10.5281/zenodo.2589280>.
- [19] K. J. Piczak, “ESC: Dataset for environmental sound classification,” in *Proc. ACM Int. Conf. Multimedia*, 2015, pp. 1015–1018.